

Cloud Programming and Software Environments

Parallel Computing and Programming Enviroments

- MapReduce
- Hadoop
- Amazon Web Services

What is MapReduce?

- Simple data-parallel programming model
- For large-scale data processing
 - Exploits large set of commodity computers
 - Executes process in distributed manner
 - Offers high availability
- Pioneered by Google
 - Processes 20 petabytes of data per day
- Popularized by open-source Hadoop project
 - Used at Yahoo!, Facebook, Amazon, ...

What is MapReduce used for?

- At Google:
 - Index construction for Google Search
 - Article clustering for Google News
 - Statistical machine translation
- At Yahoo!:
 - “Web map” powering Yahoo! Search
 - Spam detection for Yahoo! Mail
- At Facebook:
 - Data mining
 - Ad optimization
 - Spam detection

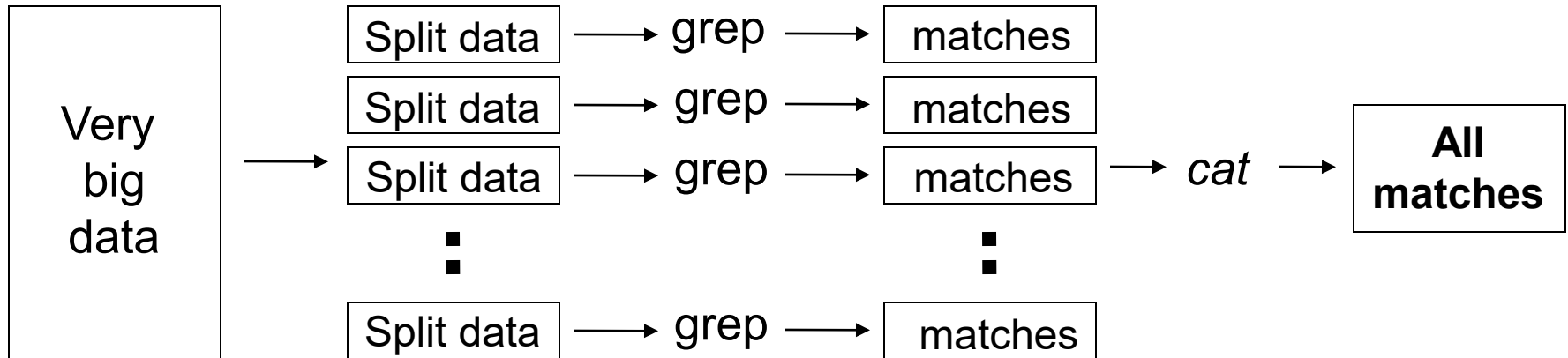
Motivation: Large Scale Data Processing

- Many tasks composed of processing lots of data to produce lots of other data
- Want to use hundreds or thousands of CPUs
... but this needs to be easy!
- MapReduce provides
 - User-defined functions
 - Automatic parallelization and distribution
 - Fault-tolerance
 - I/O scheduling
 - Status and monitoring

What is MapReduce used for?

- In research:
 - Astronomical image analysis (Washington)
 - Bioinformatics (Maryland)
 - Analyzing Wikipedia conflicts (PARC)
 - Natural language processing (CMU)
 - Particle physics (Nebraska)
 - Ocean climate simulation (Washington)
 - <Your application here>

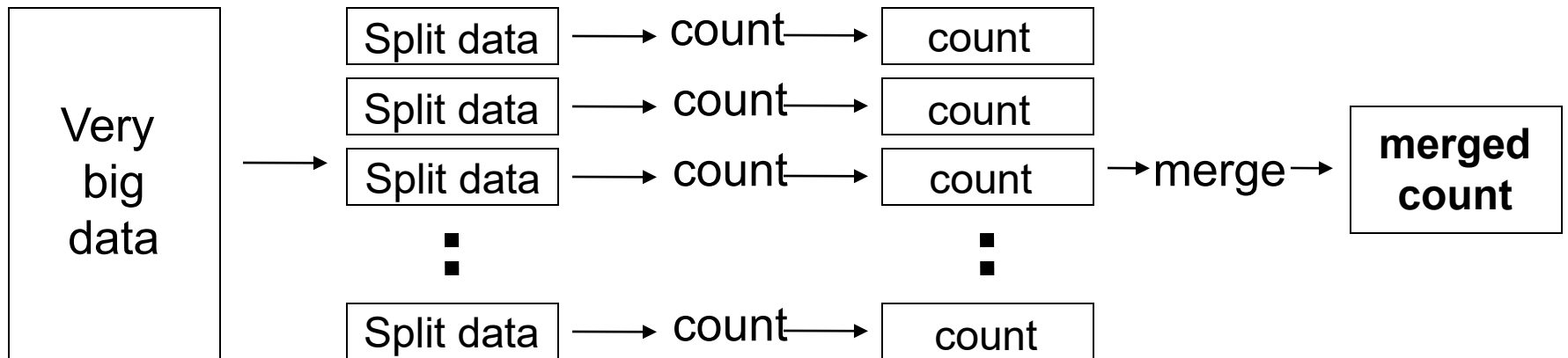
Distributed Grep



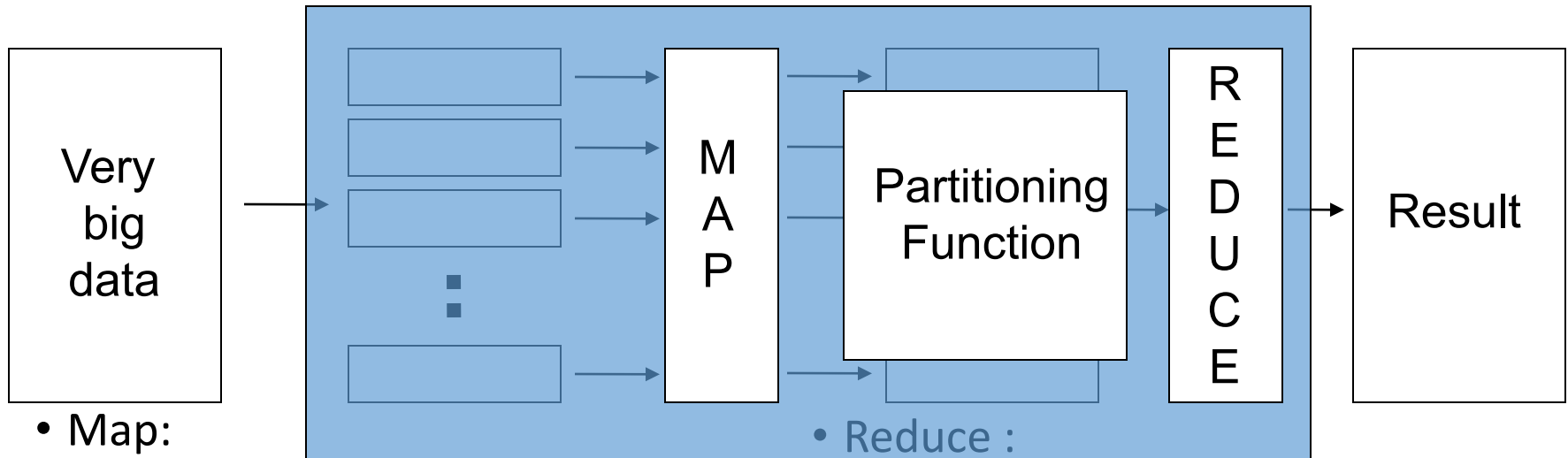
grep is a command-line utility for searching plain-text data sets for lines matching a regular expression.

cat is a standard Unix utility that concatenates and lists files

Distributed Word Count



Map+Reduce



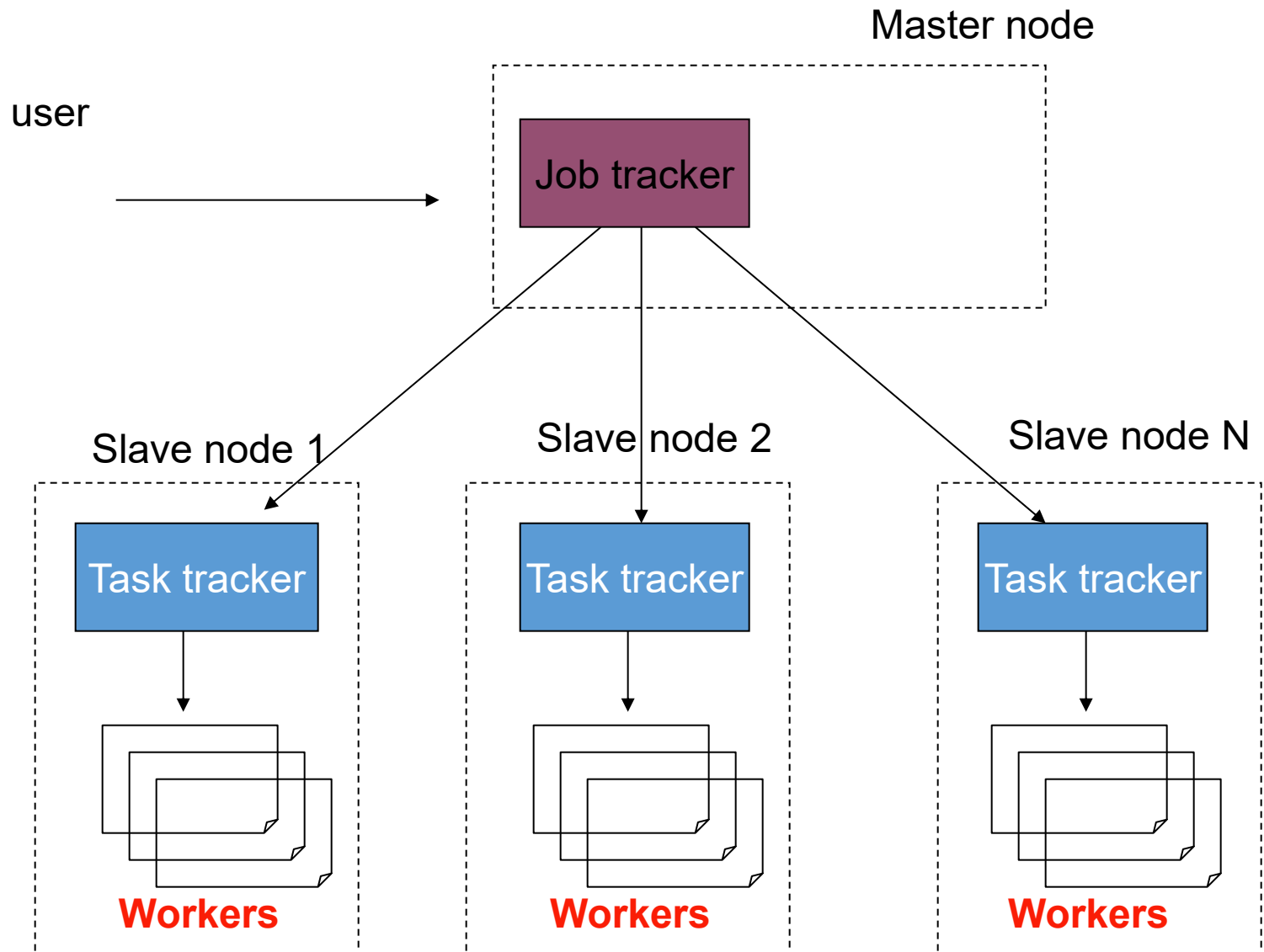
- Map:

- Accepts *input* key/value pair
- Emits *intermediate* key/value pair

- Reduce :

- Accepts *intermediate* key/value* pair
- Emits *output* key/value pair

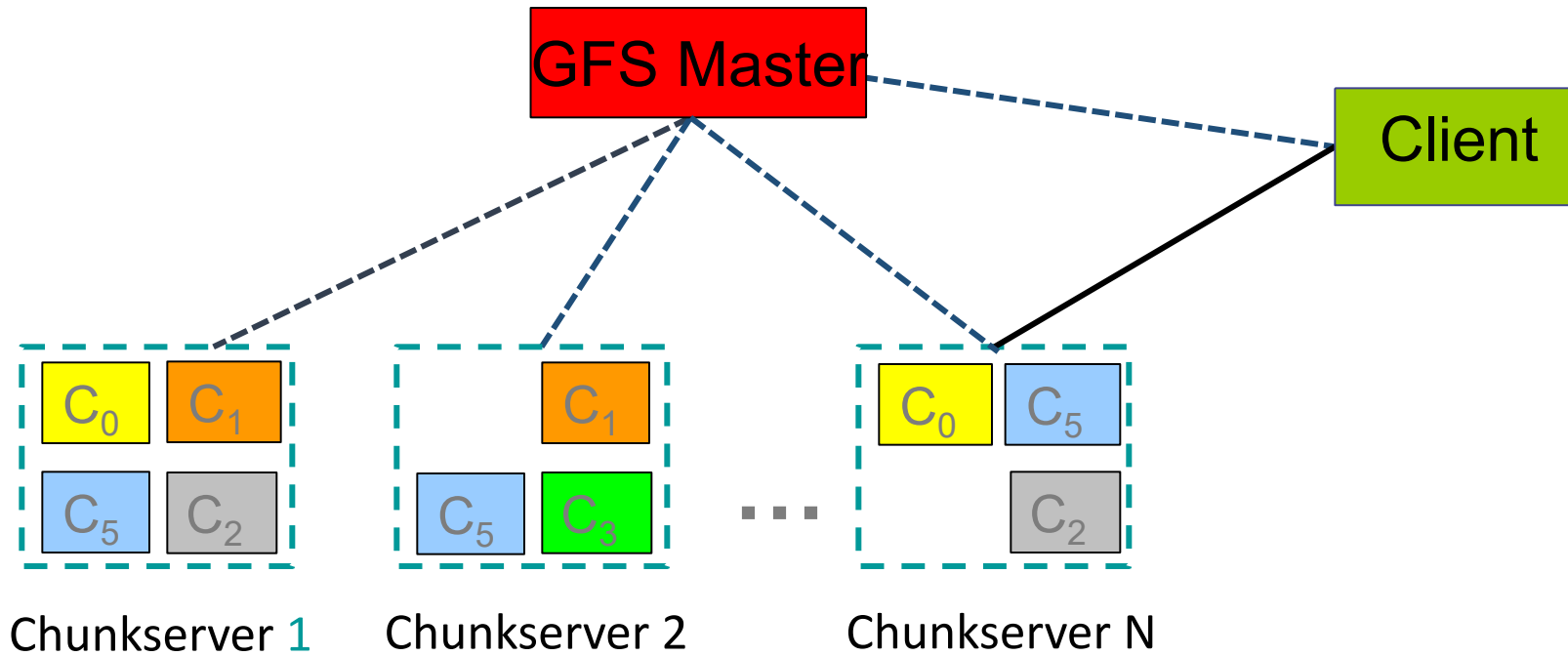
Architecture overview



GFS: underlying storage system

- Goal
 - global view
 - make huge files available in the face of node failures
- Master Node (meta server)
 - Centralized, index all chunks on data servers
- Chunk server (data server)
 - File is split into contiguous chunks, typically 16-64MB.
 - Each chunk replicated (usually 2x or 3x).
 - Try to keep replicas in different racks.

GFS architecture



Functions in the Model

- Map
 - Process a key/value pair to generate intermediate key/value pairs
- Reduce
 - Merge all intermediate values associated with the same key
- Partition
 - By default : $\text{hash}(\text{key}) \bmod R$
 - Well balanced

Programming Concept

- Map

- **Perform** a function on **individual values** in a data set to create a **new list** of values
- Example: $x = x * x$
map square [1,2,3,4,5]
returns [1,4,9,16,25]

- Reduce

- **Combine** values in a data set to create a new **value**
- Example: sum = (each elem in arr, total +=)
reduce [1,2,3,4,5]
returns 15 (the sum of the elements)

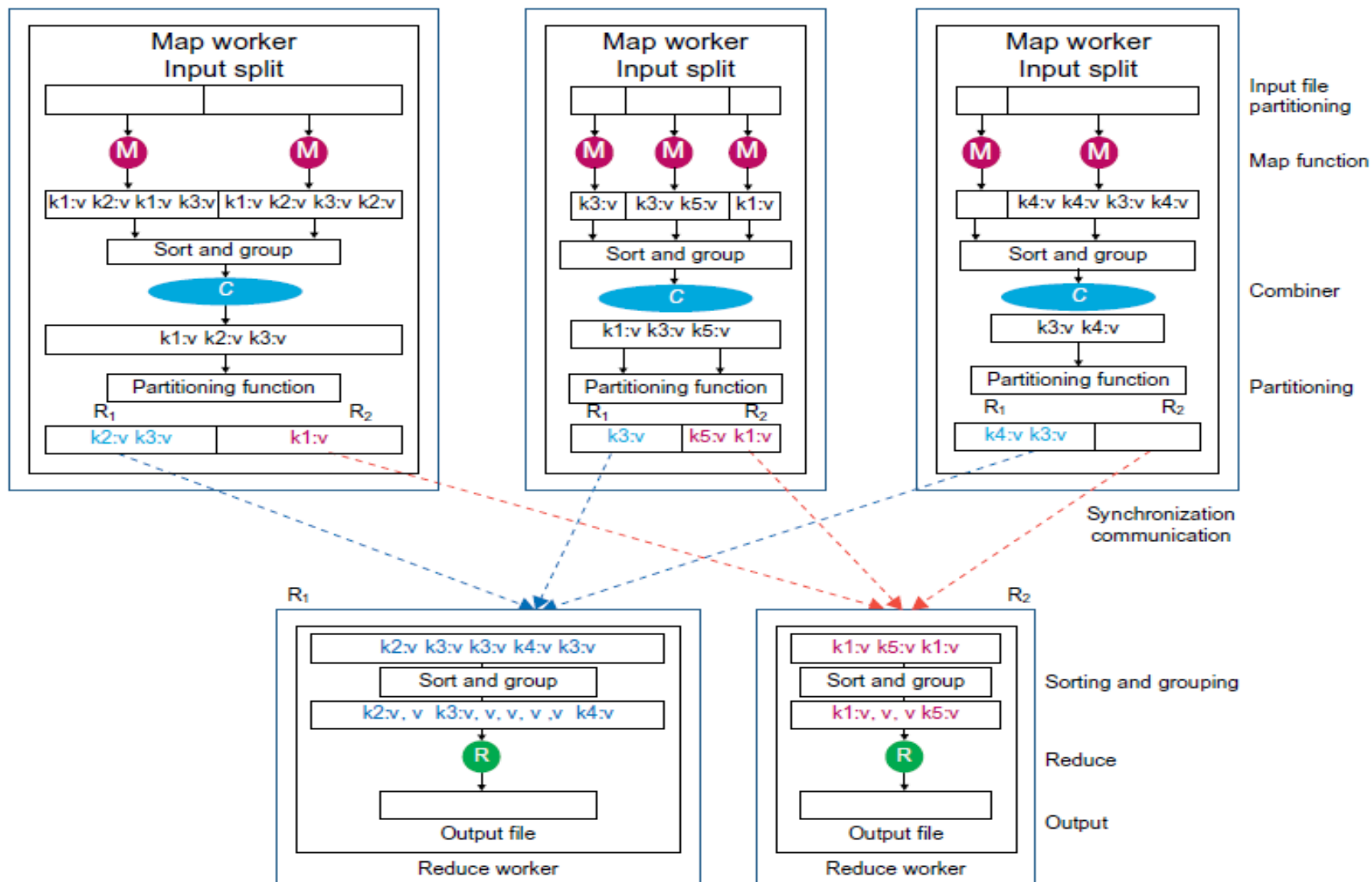


FIGURE 6.5

Data flow implementation of many functions in the Map workers and in the Reduce workers through multiple sequences of partitioning, combining, synchronization and communication, sorting and grouping, and reduce operations.

A Simple Example

- Counting words in a large set of documents

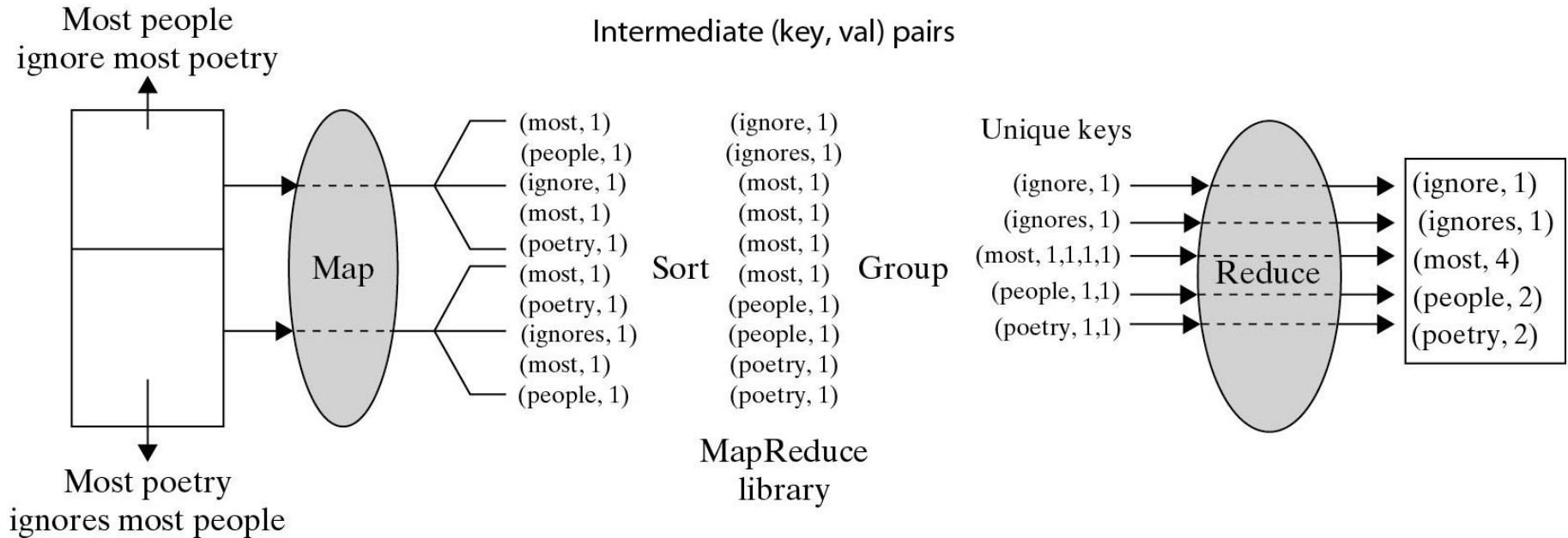
```
map(string value)
//key: document name
//value: document contents
for each word w in value
    EmitIntermediate(w, "1");
```

The **map** function emits each word w plus an associated count of occurrences (just a “1” is recorded in this pseudo-code)

```
reduce(string key, iterator values)
//key: word
//values: list of counts
int results = 0;
for each v in values
    result += ParseInt(v);
Emit(AsString(result));
```

The **reduce** function sums together all counts emitted for a particular word

A Word Counting Example on <Key, Count> Distribution



How Does it work?

- Map invocations are distributed across multiple machines by automatically partitioning the input data into a set of M splits.
- Reduce invocations are distributed by partitioning the intermediate key space into R pieces using a hash function: $\text{hash}(\text{key}) \bmod R$.
 - R and the partitioning function are specified by the programmer.

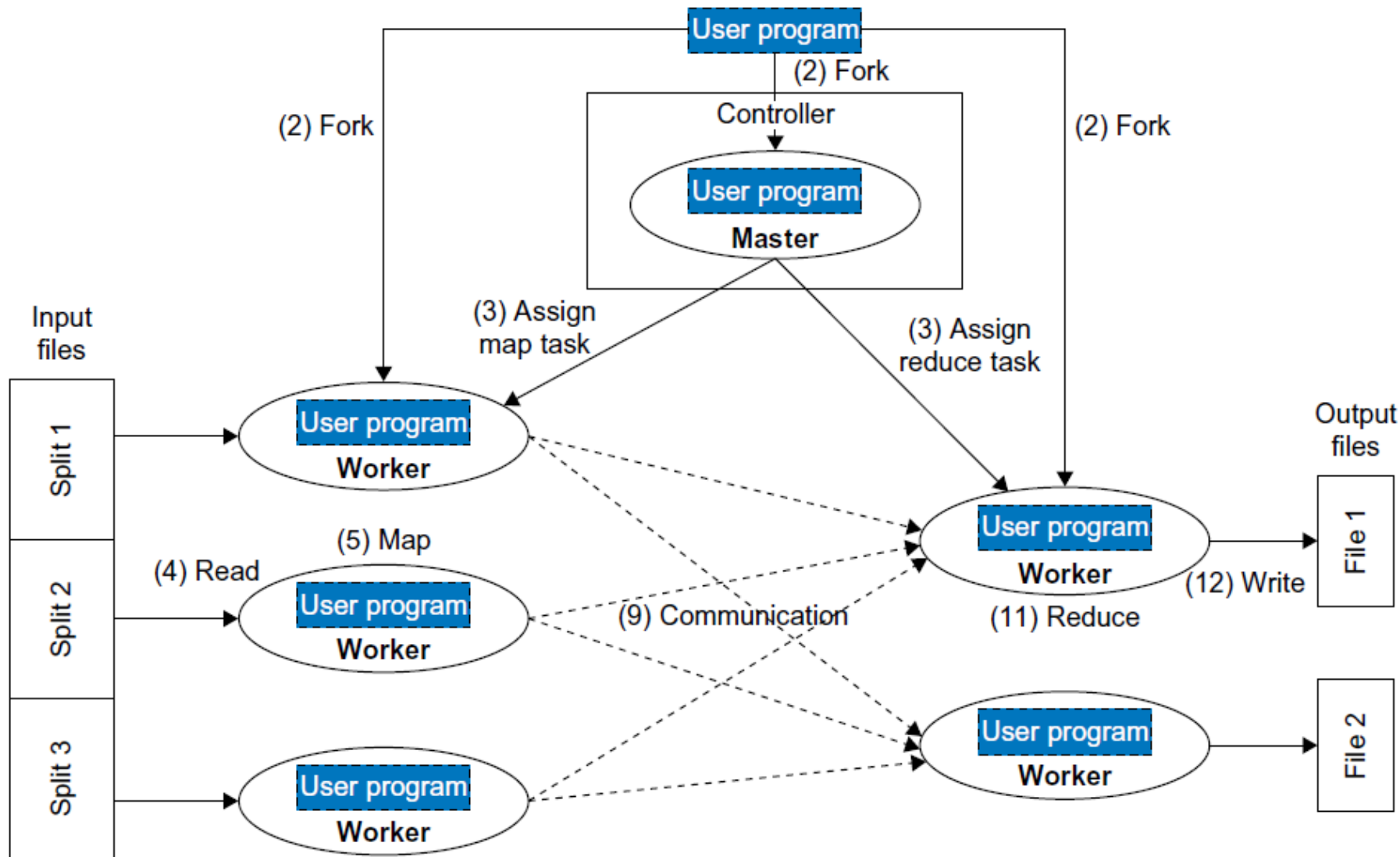


FIGURE 6.6

Control flow implementation of the MapReduce functionalities in Map workers and Reduce workers (running user programs) from input files to the output files under the control of the master user program.

MapReduce : Operation Steps

When the user program calls the MapReduce function, the following sequence of actions occurs :

- 1) The MapReduce library in the user program first splits the input files into M pieces – 16 megabytes to 64 megabytes (MB) per piece. It then starts up many copies of program on a cluster of machines.
- 2) One of the copies of program is master. The rest are workers that are assigned work by the master.

MapReduce : Operation Steps

3) A worker who is assigned a map task :

- reads the contents of the corresponding input split
- parses key/value pairs out of the input data and passes each pair to the user - defined Map function.

The intermediate key/value pairs produced by the Map function are buffered in memory.

4) The buffered pairs are written to local disk, partitioned into R regions by the partitioning function.

The location of these buffered pairs on the local disk are passed back to the master, who forwards these locations to the reduce workers.

MapReduce : Operation Steps

- 5) When a reduce worker is notified by the master about these locations, it reads the buffered data from the local disks of the map workers.

When a reduce worker has read all intermediate data, it sorts it by the intermediate keys so that all occurrences of the same key are grouped together.

- 6) The reduce worker iterates over the sorted intermediate data and for each unique intermediate key, it passes the key and the corresponding set of intermediate values to the user's Reduce function.

The output of the Reduce function is appended to a final output file.

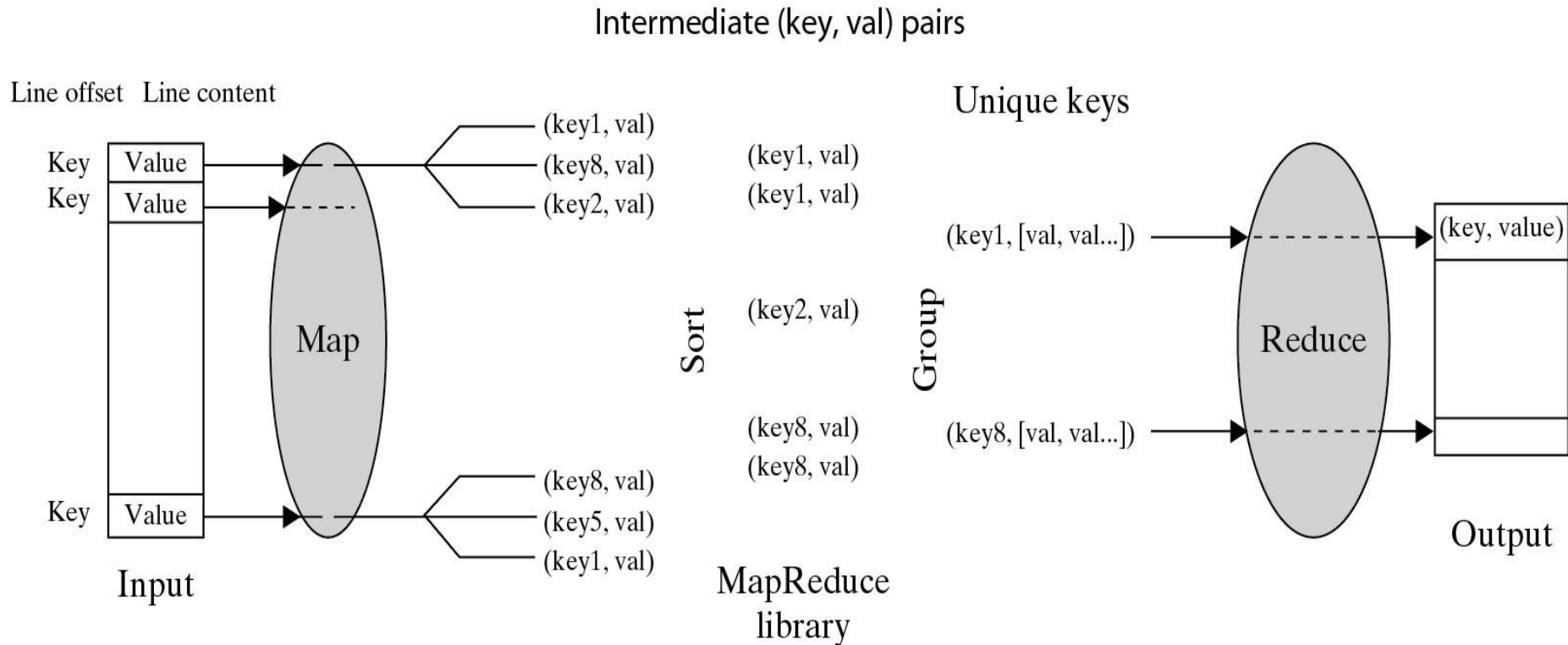
MapReduce : Operation Steps

7) When all map tasks and reduce tasks have been completed, the master wakes up the user program.

At this point, MapReduce call in the user program returns back to the user code.

After successful completion, output of the mapreduce execution is available in the R output files.

Logical Data Flow in 5 Processing Steps in MapReduce Process



(Key, Value) Pairs are generated by the Map function over multiple available Map Workers (VM instances). These pairs are then sorted and group based on key ordering. Different key-groups are then processed by multiple Reduce Workers in parallel.

Locality issue

- Master scheduling policy
 - Asks GFS for locations of replicas of input file blocks
 - Map tasks typically split into 64MB (== GFS block size)
 - Map tasks scheduled so GFS input block replica are on same machine or same rack
- Effect
 - Thousands of machines read input at local disk speed
 - Without this, rack switches limit read rate

Fault Tolerance

- Reactive way

- Worker failure

- Heartbeat, Workers are periodically pinged by master
 - NO response = failed worker
 - If the processor of a worker fails, the tasks of that worker are reassigned to another worker.

- Master failure

- Master writes periodic checkpoints
 - Another master can be started from the last checkpointed state
 - If eventually the master dies, the job will be aborted

Fault Tolerance

- Proactive way (**Redundant Execution**)
 - The problem of “stragglers” (slow workers)
 - Other jobs consuming resources on machine
 - Bad disks with soft errors transfer data very slowly
 - Weird things: processor caches disabled (!!)
 - When computation almost done, reschedule in-progress tasks
 - Whenever either the primary or the backup executions finishes, mark it as completed

Fault Tolerance

- Input error: bad records
 - Map/Reduce functions sometimes fail for particular inputs
 - Best solution is to debug & fix, but not always possible
- On segment fault
 - Send UDP packet to master from signal handler
 - Include sequence number of record being processed
- Skip bad records
 - If master sees two failures for same record, next worker is told to skip the record

Points need to be emphasized

- No *reduce* can begin until *map* is complete
- Master must communicate locations of intermediate files
- Tasks scheduled based on location of data
- If *map* worker fails any time before *reduce* finishes, task must be completely rerun
- MapReduce library does most of the hard work for us!

Other Examples

- Distributed Grep:
 - Map function emits a line if it matches a supplied pattern.
 - Reduce function is an identity function that copies the supplied intermediate data to the output.
- Count of URL accesses:
 - Map function processes logs of web page requests and outputs <URL, 1>.
 - Reduce function adds together all values for the same URL, emitting <URL, total count> pairs.
- Reverse Web-Link graph; e.g., all URLs with reference to <http://dblab.usc.edu>:
 - Map function outputs <tgt, src> for each link to a tgt in a page named src.
 - Reduce concatenates the list of all src URLs associated with a given tgt URL and emits the pair: <tgt, list(src)>.
- Inverted Index; e.g., all URLs with 585 as a word:
 - Map function parses each document, emitting a sequence of <word, doc_ID>.
 - Reduce accepts all pairs for a given word, sorts the corresponding doc_IDs and emits a <word, list(doc_ID)> pair.
 - Set of all output pairs forms a simple inverted index.

MapReduce Implementations

MapReduce



Cluster,
1, Google
2, Apache Hadoop



Multicore CPU,
Phoenix @ stanford



GPU,
Mars@HKUST

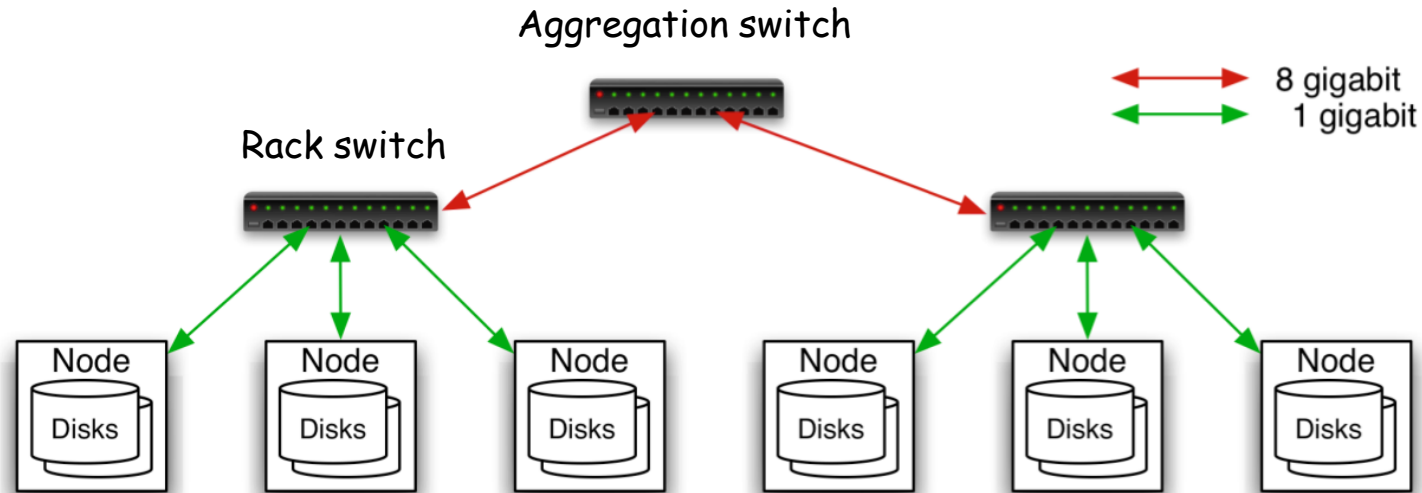
Hadoop

Software platform originally developed by Yahoo enabling users to write and run applications over vast distributed data.

Attractive Features in Hadoop :

- **Scalable** : can easily scale to store and process petabytes of data in the Web space
- **Economical** : An open-source MapReduce minimizes the overheads in task spawning and massive data communication.
- **Efficient**: Processing data with high-degree of parallelism across a large number of commodity nodes
- **Reliable** : Automatically maintains multiple copies of data to facilitate redeployment of computing tasks on failures

Typical Hadoop Cluster



- 40 nodes/rack, 1000-4000 nodes in cluster
- 1 Gbps bandwidth within rack, 8 Gbps out of rack
- Node specs (Yahoo terasort):
8 x 2GHz cores, 8 GB RAM, 4 disks (= 4 TB?)

Typical Hadoop Cluster



Challenges

Cheap nodes fail, especially if you have many

1. Mean time between failures for 1 node = 3 years
2. Mean time between failures for 1000 nodes = 1 day
3. Solution: Build fault-tolerance into system

Commodity network = low bandwidth

1. Solution: Push computation to the data

Programming distributed systems is hard

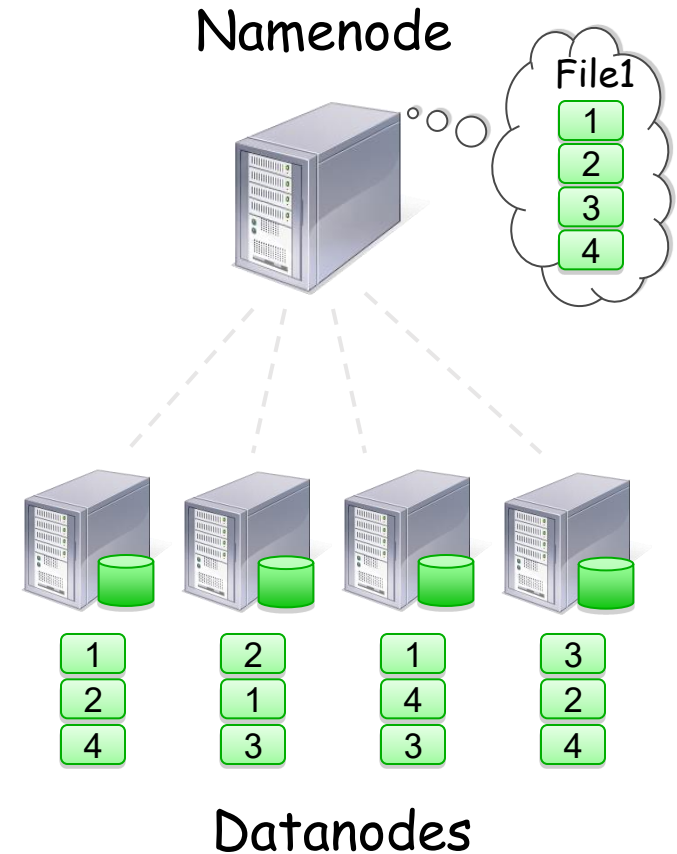
1. Solution: Data-parallel programming model: users write “map” & “reduce” functions, system distributes work and handles faults

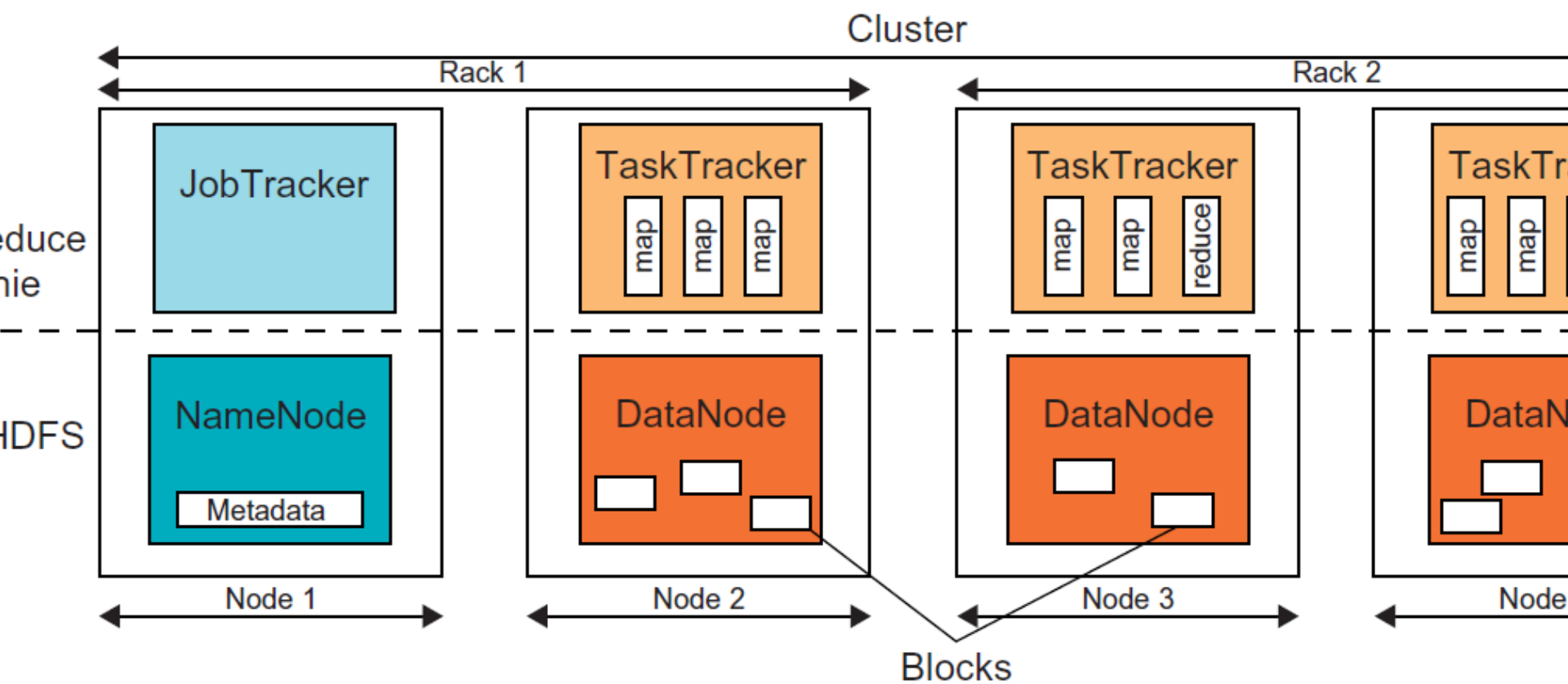
Hadoop Components

- Distributed file system (HDFS)
 - Single namespace for entire cluster
 - Replicates data 3x for fault-tolerance
- MapReduce framework
 - Executes user jobs specified as “map” and “reduce” functions
 - Manages work distribution & fault-tolerance

Hadoop Distributed File System

- Files split into 128MB *blocks*
- Blocks replicated across several *datanodes* (usually 3)
- Single *namenode* stores metadata (file names, block locations, etc)
- Optimized for large files, sequential reads
- Files are append-only





.11

and MapReduce architecture in Hadoop where boxes with different shadings refer to different nodes applied to different blocks of data.

Secure Query Processing with Hadoop/MapReduce

- Query Rewriting and Optimization Principles defined and implemented for two types of data
 - (i) Relational data: Secure query processing with HIVE
 - (ii) RDF Data: Secure query processing with SPARQL
- Demonstrated with XACML Policies (content, temporal, association)
- Joint demonstration with Kings College and U. of Insubria
 - First demo (2010): Each party submits their data and policies
 - Our cloud will manage the data and policies
 - Second demo (2011): Multiple clouds

Higher-level languages over Hadoop: Pig and Hive

Motivation

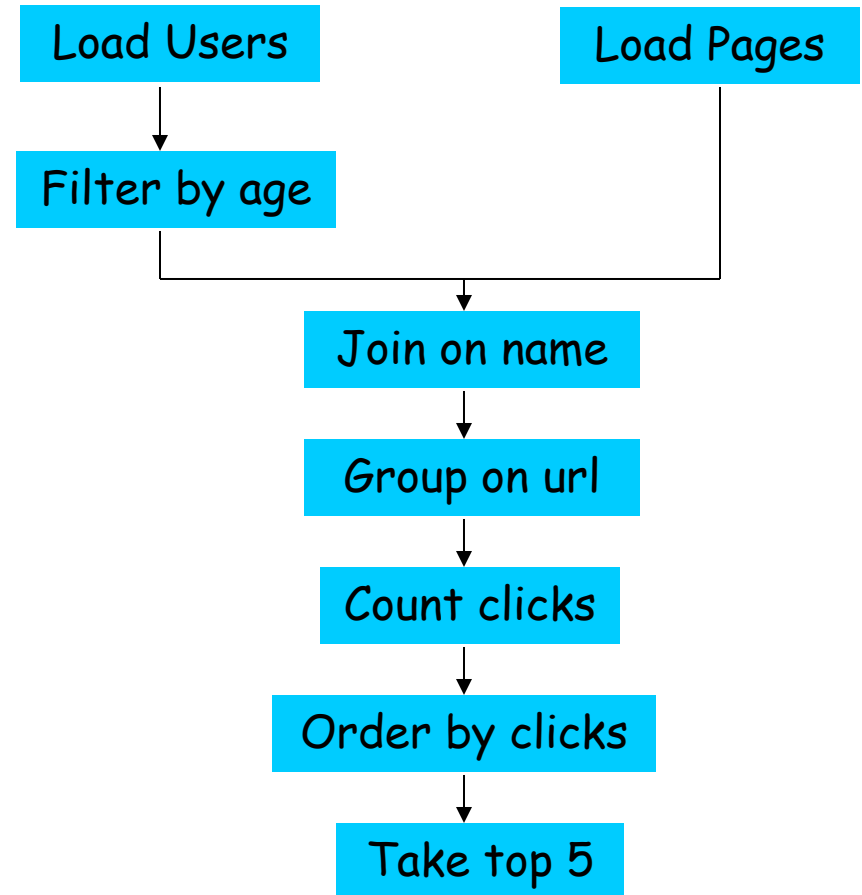
- Many parallel algorithms can be expressed by a series of MapReduce jobs
- But MapReduce is fairly low-level: must think about keys, values, partitioning, etc
- Can we capture common “job building blocks”?

Pig

- Started at Yahoo! Research
- Runs about 30% of Yahoo!'s jobs
- Features:
 - Expresses sequences of MapReduce jobs
 - Data model: nested “bags” of items
 - Provides relational (SQL) operators (JOIN, GROUP BY, etc)
 - Easy to plug in Java functions
 - Pig Pen development environment for Eclipse

An Example Problem

Suppose you have user data in one file, page view data in another, and you need to find the top 5 most visited pages by users aged 18 - 25.



In MapReduce

```
import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.Writable;
import org.apache.hadoop.io.WritableComparable;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.mapred.KeyValueTextInputFormat;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Recorder;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;
import org.apache.hadoop.mapred.SequenceFileInputFormat;
import org.apache.hadoop.mapred.SequenceFileOutputFormat;
import org.apache.hadoop.mapred.TextInputFormat;
import org.apache.hadoop.mapred.JobControl.Job;
import org.apache.hadoop.mapred.lib.IdentityMapper;

public class MRExample {
    public static class LoadPages extends MapReduceBase
        implements Mapper<LongWritable, Text, Text, Text> {

        public void map(LongWritable k, Text val,
            OutputCollector<Text, Text> oc,
            Reporter reporter) throws IOException {
            // Pull the key out
            String line = val.toString();
            int firstComma = line.indexOf(',');
            String key = line.substring(0, firstComma);
            String value = line.substring(firstComma + 1);
            Text outKey = new Text(key);
            // Prepend an index to the value so we know which file
            // it came from.
            Text outVal = new Text(1 + value);
            oc.collect(outKey, outVal);
        }

        public static class LoadAndFilterUsers extends MapReduceBase
            implements Mapper<LongWritable, Text, Text, Text> {

        public void map(LongWritable k, Text val,
            OutputCollector<Text, Text> oc,
            Reporter reporter) throws IOException {
            // Pull the key out
            String line = val.toString();
            int firstComma = line.indexOf(',');
            String value = line.substring(firstComma + 1);
            int age = Integer.parseInt(value);
            if (age < 18 || age > 25) return;
            String key = line.substring(0, firstComma);
            Text outKey = new Text(key);
            // Prepend an index to the key so we know which file
            // it came from.
            Text outVal = new Text("2" + value);
            oc.collect(outKey, outVal);
        }

        public static class Join extends MapReduceBase
            implements Reducer<Text, Text, Text, Text> {

        public void reduce(Text key,
            Iterator<Text> iter,
            OutputCollector<Text, Text> oc,
            Reporter reporter) throws IOException {
            // For each value, figure out which file it's from and
            // accordingly.
            List<String> first = new ArrayList<String>();
            List<String> second = new ArrayList<String>();

            while (iter.hasNext()) {
                Text t = iter.next();
                String val = t.toString();
                if (value.charAt(0) == '1')
                    first.add(value.substring(1));
                else second.add(value.substring(1));
            }

            reporter.setStatus("OK");
        }

        // Do the cross product and collect the values
        for (String s1 : first) {
            for (String s2 : second) {
                String outval = key + "," + s1 + "," + s2;
                oc.collect(null, new Text(outval));
                reporter.setStatus("OK");
            }
        }

        public static class LoadJoined extends MapReduceBase
            implements Mapper<Text, Text, LongWritable> {

        public void map(
            Text k,
            Text val,
            OutputCollector<Text, LongWritable> oc,
            Reporter reporter) throws IOException {
            // Find the url
            String line = val.toString();
            int firstComma = line.indexOf(',');
            int secondComma = line.indexOf(',', firstComma + 1);
            String key = line.substring(firstComma, secondComma);
            // drop the rest of the record, I don't need it anymore
            // just pass a 1 for the combiner/reducer to sum
            Text outKey = new Text(key);
            oc.collect(outKey, new LongWritable(1L));
        }

        public static class ReduceUrls extends MapReduceBase
            implements Reducer<Text, LongWritable, WritableComparable, Writable> {

        public void reduce(
            Text k,
            Iterator<LongWritable> iter,
            OutputCollector<WritableComparable, Writable> oc,
            Reporter reporter) throws IOException {
            // Add up all the values we see
            long sum = 0;
            while (iter.hasNext()) {
                sum += iter.next().get();
                reporter.setStatus("OK");
            }

            oc.collect(key, new LongWritable(sum));
        }

        public static class LoadClicks extends MapReduceBase
            implements Mapper<WritableComparable, Writable, LongWritable, Text> {

        public void map(
            WritableComparable key,
            Writable val,
            OutputCollector<LongWritable, Text> oc,
            Reporter reporter) throws IOException {
            oc.collect((LongWritable) val, (Text) key);
        }

        public static class LimitClicks extends MapReduceBase
            implements Reducer<LongWritable, Text, LongWritable, Text> {

        int count = 0;
        public void reduce(
            LongWritable key,
            Iterator<Text> iter,
            OutputCollector<LongWritable, Text> oc,
            Reporter reporter) throws IOException {
            // Only output the first 100 records
            while (count < 100 && iter.hasNext()) {
                oc.collect(key, iter.next());
                count++;
            }

            public static void main(String[] args) throws IOException {
                JobConf lp = new JobConf(MRExample.class);
                lp.setJobName("Load Pages");
                lp.setInputFormat(TextInputFormat.class);
                lp.setOutputKeyClass(Text.class);
                lp.setOutputValueClass(Text.class);
                lp.setMapperClass(LoadPages.class);
                FileInputFormat.addInputPath(lp, new Path("/user/gates/pages"));
                FileOutputFormat.setOutputPath(lp, new Path("/user/gates/tmp/indexed_pages"));
                lp.setNumReduceTasks(0);
                Job loadPages = new Job(lp);

                JobConf lfu = new JobConf(MRExample.class);
                lfu.setJobName("Load and Filter Users");
                lfu.setInputFormat(TextInputFormat.class);
                lfu.setOutputKeyClass(Text.class);
                lfu.setOutputValueClass(Text.class);
                lfu.setMapperClass(LoadAndFilterUsers.class);
                FileInputFormat.addInputPath(lfu, new Path("/user/gates/users"));
                FileOutputFormat.setOutputPath(lfu, new Path("/user/gates/tmp/filtered_users"));
                lfu.setNumReduceTasks(0);
                Job loadUsers = new Job(lfu);

                JobConf join = new JobConf(MRExample.class);
                join.setJobName("Join Users and Pages");
                join.setInputFormat(TextInputFormat.class);
                join.setOutputKeyClass(Text.class);
                join.setOutputValueClass(Text.class);
                join.setMapperClass(LoadPages.class);
                join.setReducerClass(Join.class);
                FileInputFormat.addInputPath(join, new Path("/user/gates/tmp/indexed_pages"));
                FileInputFormat.addInputPath(join, new Path("/user/gates/tmp/filtered_users"));
                FileOutputFormat.setOutputPath(join, new Path("/user/gates/tmp/joined"));
                join.setNumReduceTasks(50);
                Job joinJob = new Job(join);
                joinJob.addDependingJob(loadPages);
                joinJob.addDependingJob(loadUsers);

                JobConf group = new JobConf(MRExample.class);
                group.setJobName("Group URLs");
                group.setInputFormat(KeyValueTextInputFormat.class);
                group.setOutputKeyClass(Text.class);
                group.setOutputValueClass(LongWritable.class);
                group.setInputFormat(SequenceFileInputFormat.class);
                group.setMapperClass(LoadJoined.class);
                group.setCombinerClass(ReduceUrls.class);
                group.setReducerClass(ReduceUrls.class);
                FileInputFormat.addInputPath(group, new Path("/user/gates/tmp/joined"));
                FileOutputFormat.setOutputPath(group, new Path("/user/gates/tmp/grouped"));
                group.setNumReduceTasks(50);
                Job groupJob = new Job(group);
                groupJob.addDependingJob(joinJob);

                JobConf top100 = new JobConf(MRExample.class);
                top100.setJobName("Top 100 sites");
                top100.setInputFormat(SequenceFileInputFormat.class);
                top100.setOutputKeyClass(LongWritable.class);
                top100.setOutputValueClass(Text.class);
                top100.setInputFormat(SequenceFileInputFormat.class);
                top100.setMapperClass(LoadClicks.class);
                top100.setCombinerClass(LimitClicks.class);
                top100.setReducerClass(LimitClicks.class);
                FileInputFormat.addInputPath(top100, new Path("/user/gates/tmp/grouped"));
                FileOutputFormat.setOutputPath(top100, new Path("/user/gates/top100sitesforusers18to25"));
                top100.setNumReduceTasks(1);
                Job limit = new Job(top100);
                limit.addDependingJob(groupJob);

                JobControl jc = new JobControl(MRExample.class);
                jc.addJob(loadPages);
                jc.addJob(loadUsers);
                jc.addJob(joinJob);
                jc.addJob(groupJob);
                jc.addJob(limit);
                jc.run();
            }
        }
    }
}
```

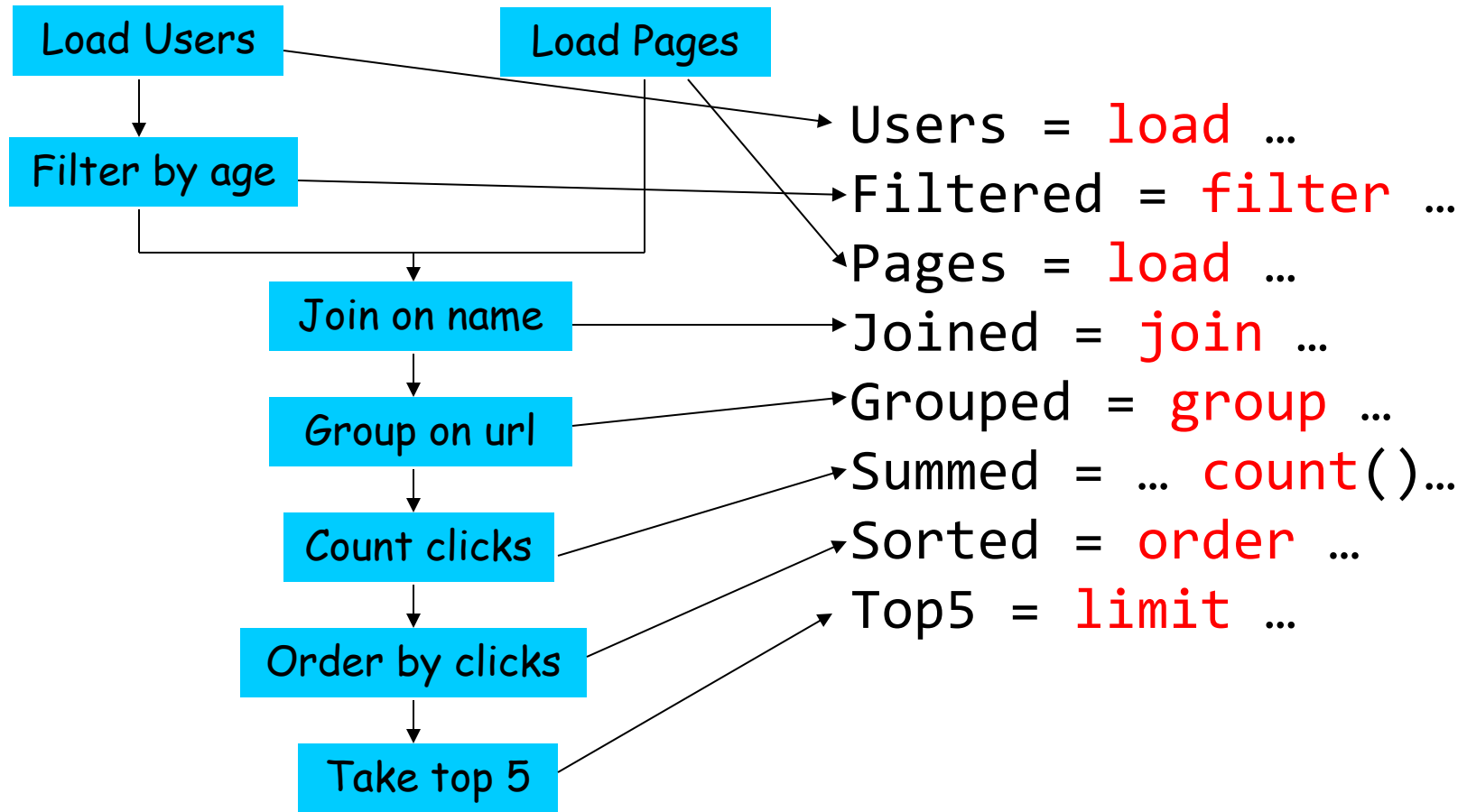
In Pig Latin

```
Users      = load 'users' as (name, age);
Filtered   = filter Users by
              age >= 18 and age <= 25;
Pages      = load 'pages' as (user, url);
Joined     = join Filtered by name, Pages by user;
Grouped    = group Joined by url;
Summed     = foreach Grouped generate group,
              count(Joined) as clicks;
Sorted     = order Summed by clicks desc;
Top5       = limit Sorted 5;

store Top5 into 'top5sites';
```

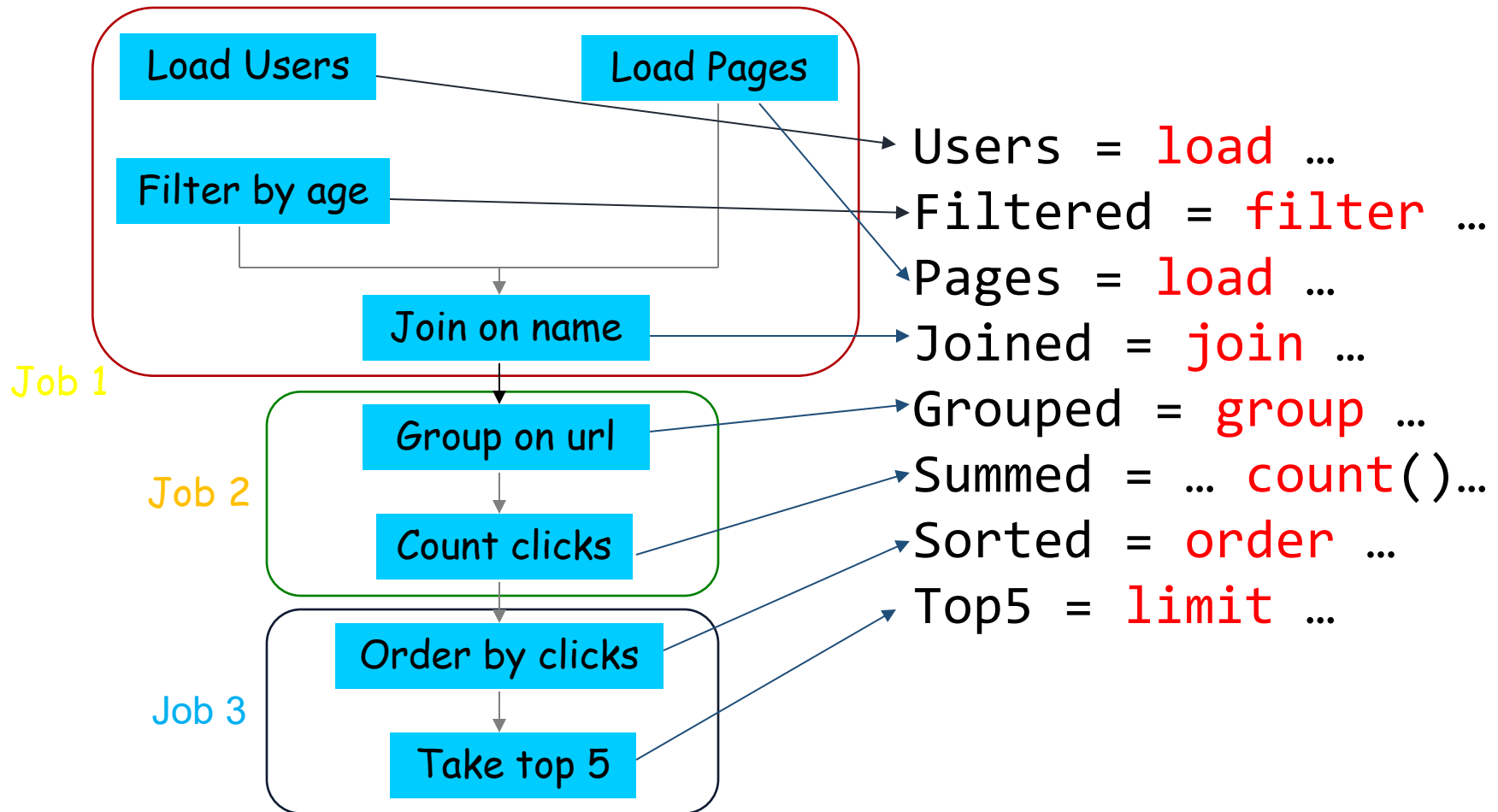
Ease of Translation

Notice how naturally the components of the job translate into Pig Latin.



Ease of Translation

Notice how naturally the components of the job translate into Pig Latin.



Hive

- Developed at Facebook
- Used for majority of Facebook jobs
- “Relational database” built on Hadoop
 - Maintains list of table schemas
 - SQL-like query language (HQL)
 - Can call Hadoop Streaming scripts from HQL
 - Supports table partitioning, clustering, complex data types, some optimizations

Sample Hive Queries

- Find top 5 pages visited by users aged 18-25:

```
SELECT p.url, COUNT(1) as clicks
FROM users u JOIN page_views p ON (u.name = p.user)
WHERE u.age >= 18 AND u.age <= 25
GROUP BY p.url
ORDER BY clicks
LIMIT 5;
```

- Filter page views through Python script:

```
SELECT TRANSFORM(p.user, p.date)
USING 'map_script.py'
AS dt, uid CLUSTER BY dt
FROM page_views p;
```

Amazon Elastic MapReduce

- Provides a web-based interface and command-line tools for running Hadoop jobs on Amazon EC2
 - Data stored in Amazon S3
 - Monitors job and shuts down machines after use
 - Small extra charge on top of EC2 pricing
-
- If you want more control over how you Hadoop runs, you can launch a Hadoop cluster on EC2 manually using the scripts in `src/contrib/ec2`

Elastic MapReduce Workflow

Create a New Job Flow

Cancel 

DEFINE JOB FLOW

SPECIFY PARAMETERS

CONFIGURE EC2 INSTANCES

REVIEW

Creating a job flow to process your data using Amazon Elastic MapReduce is simple and quick. Let's begin by giving your job flow a name and selecting its type. If you don't already have an application you'd like to run on Amazon Elastic MapReduce, samples are available to help you get started.

Job Flow Name*:

The name can be anything you like and doesn't need to be unique. It's a good idea to name the job flow something descriptive.

Type*: ☒ Streaming

A Streaming job flow allows you to write single-step mapper and reducer functions in a language other than java.

☐ Custom Jar (advanced)

A custom jar on the other hand gives you more complete control over the function of Hadoop but must be a compiled java program. Amazon Elastic MapReduce supports custom jars developed for Hadoop 0.18.3.

☐ Pig Program

Pig is a SQL-like language built on top of Hadoop. This option allows you to define a job flow that runs a Pig script, or set up a job flow that can be used interactively via SSH to run Pig commands.

☐ Sample Applications

Select a sample application and click Continue. Subsequent forms will be filled with the necessary data to create a sample Job Flow.

Word Count (Streaming)



Word count is a Python application that counts occurrences of each word in provided documents. [Learn more and view license](#)

Continue



* Required field

Elastic MapReduce Workflow

Create a New Job Flow

Cancel 

DEFINE JOB FLOW

SPECIFY PARAMETERS

CONFIGURE EC2 INSTANCES

REVIEW

Specify Mapper and Reducer functions to run within the Job Flow. The mapper and reducers may be either (i) class names referring to a mapper or reducer class in Hadoop or (ii) locations in Amazon S3. ([Click Here](#) for a list of available tools to help you upload and download files from Amazon S3.) The format for specifying a location in Amazon S3 is bucket_name/path_name. The location should point to an executable program, for example a python program. Extra arguments are passed to the Hadoop streaming program and can specify things such as additional files to be loaded into the distributed cache.

Input Location*:

The URL of the Amazon S3 Bucket that contains the input files.

Output Location*:

The URL of the Amazon S3 Bucket to store output files. Should be unique.

Mapper*:

The mapper Amazon s3 location or streaming command to execute.

Reducer*:

The reducer Amazon s3 location or streaming command to execute.

Extra Args:

[< Back](#)

Continue



* Required field

Elastic MapReduce Workflow

Create a New Job FlowCancel

✓

✓

DEFINE JOB FLOW SPECIFY PARAMETERS **CONFIGURE EC2 INSTANCES** REVIEW

Enter the number and type of EC2 instances you'd like to run your job flow on.

Number of Instances*:

The number of EC2 instances to run in your Hadoop cluster.
If you wish to run more than 20 instances, please complete the [limit request form](#).

Type of Instance*:

The type of EC2 instances to run in your Hadoop cluster ([learn more about instance types](#)).

⌵ Show advanced options

< BackContinue* Required field

Elastic MapReduce Workflow



[Contact Us](#) | [Create an AWS Account](#)

[About AWS](#) | [Products](#) | [Solutions](#) | [Resources](#) | [Support](#) | [Your Account](#)

[Home](#) > [Resources](#) > [AWS Management Console](#) [BETA](#) > [Amazon Elastic MapReduce](#)

Welcome, Rad Lab | [Settings](#) | [Sign Out](#)

Amazon EC2 | **Amazon Elastic MapReduce** | Amazon CloudFront

Your Elastic MapReduce Job Flows

Region: [US-East](#) [Create New Job Flow](#) [Terminate](#) [Show/Hide](#) [Refresh](#) [Help](#)

Viewing: [All](#) [1 to 1 of 1 Job Flows](#)

	Name	State	Creation Date	Elapsed Time	Normalized Instance Hours
	My Job Flow	STARTING	2009-08-19 14:50 PDT	0 hours 0 minutes	0

1 Job Flow selected

Id:	j-46JL0YQ7ZPH1	Creation Date:	2009-08-19 14:50 PDT
Name:	My Job Flow	Start Date:	-
State:	STARTING	End Date:	-
Last State Change Reason:	Starting instances		
Availability Zone:	us-east-1b	Instance Count:	4

© 2008 - 2009, Amazon Web Services LLC or its affiliates. All right reserved. | [Feedback](#) | [Support](#) | [Privacy Policy](#) | [Terms of Use](#)

Conclusions

- MapReduce programming model hides the complexity of work distribution and fault tolerance
- Principal design philosophies:
 - *Make it scalable*, so you can throw hardware at problems
 - *Make it cheap*, lowering hardware, programming and admin costs
- MapReduce is not suitable for all problems, but when it works, it may save you quite a bit of time
- Cloud computing makes it straightforward to start using Hadoop (or other parallel software) at scale

Resources

- Hadoop: <http://hadoop.apache.org/core/>
- Pig: <http://hadoop.apache.org/pig>
- Hive: <http://hadoop.apache.org/hive>
- Video tutorials: <http://www.cloudera.com/hadoop-training>

- Amazon Web Services: <http://aws.amazon.com/>
- Amazon Elastic MapReduce guide:
<http://docs.amazonwebservices.com/ElasticMapReduce/latest/GettingStartedGuide/>

Based on: Kai Hwang, University of Southern California
with additions from Matei Zaharia, EECS, UC Berkeley